PART IV — FROM RAG TO AGENTIC RAG

Chapter 19

Orchestration with LangGraph

"The moment two independent things could happen at once, a linear loop stops being able to say so."

Chapter 19 — Orchestration with LangGraph

This chapter introduces LangGraph on its own terms, independent of Memora: nodes, edges, conditional routing, a typed state object, and the two mechanisms — reducers and fan-in barriers — that make parallel branches rejoin correctly instead of racing each other. Chapter 19B then takes every primitive introduced here and ports Chapter 18's exact state machine onto it, node by node.

19.1 Why graph-based orchestration — and why not sooner

Memora did not adopt LangGraph the first time someone suggested it. The project's own architecture record shows a deliberate deferral: no parallel sub-query retrieval existed yet, no human-in-the-loop step was planned, and no durable-checkpoint requirement had appeared — three concrete triggers the team had agreed would justify the migration, none of which were true yet. A framework adopted because loops are supposedly inelegant, rather than because a real requirement needs what the framework uniquely provides, is a rewrite in search of a justification.

The deferral held until the two-track retrieval architecture (Chapter 11.5) made it concrete: `documents` and `learned_qa` compress through independent NAC/DC/LBC pipelines that share no data and never need to run in sequence relative to each other. That is a genuine parallel opportunity `run_agent`'s single-threaded loop structurally cannot express — not because Python cannot run two things at once, but because a hand-written loop has no primitive for "these two branches, then rejoin." That gap, not general dissatisfaction with loops, is what finally justified the migration.

> **Common pitfall — Migrating before the first real attempt teaches you the mechanics**
> An early, direct migration attempt built a full `TypedDict` mirror of the agent's state and a toy visualization script before anyone on the project had run a single conditional edge by hand — and immediately hit three mechanical mistakes in one sitting: a missing `__main__` guard, a Jupyter-only display call used in a terminal script, and `.get_graph()` called on an uncompiled graph. The fix was not to push through; it was to delete the production-shaped files, build a small, disposable graph wired to real project objects, and only resume the real port once the mechanics were second nature. Practice on something you can afford to get wrong before you rewrite something you can't.

19.2 Nodes, edges, and conditional edges — the three primitives

> **Definition — Node**
> A plain function that accepts the graph's state and returns a partial update — a dictionary of the fields it changed, not the whole state object — registered under a string name with `graph.add_node(name, fn)`.

> **Definition — Edge**
> A fixed transition from one named node to another (or to `END`), declared with `graph.add_edge(a, b)`, that always fires when node `a` finishes — no decision involved.

Definition — Conditional edge
A transition whose destination is computed at runtime by a routing function that reads the current state and returns the name of the next node, declared with `graph.add_conditional_edges(name, routing_fn, {label: target, ...})`.

```
graph = StateGraph(GraphState)
graph.add_node("decide", decide)
graph.add_node("retrieve", retrieve)
graph.add_node("generate", generate)

graph.add_edge(START, "decide")
graph.add_conditional_edges(
    "decide", route_decide,
    {"retrieve": "retrieve", "generate": "generate"},
)
graph.add_edge("retrieve", "decide")    # loop back
graph.add_edge("generate", END)
```

Three primitives, and nothing else is load-bearing. A node changes state; an edge says what runs next unconditionally; a conditional edge asks a small routing function to decide. Every graph in this book — Memora's real one included — is built from nothing more than these three calls, repeated.

19.3 The GraphState TypedDict

Definition — GraphState
A `TypedDict` declaring every field any node in the graph may read or write, given once to `StateGraph(GraphState)` so the framework knows the complete shape of the state it is passing between nodes.

Every field a graph will ever touch is declared once, up front — not accumulated implicitly the way a hand-written loop's local variables are. A node cannot silently invent a new piece of state; if it is not in `GraphState`, it does not exist as far as the graph is concerned.

```
class GraphState(TypedDict):
    query: str
    query_variants: list[str]
    retrieved_document_chunks: Annotated[list[dict], operator.add]
    retrieved_learned_qa_chunks: Annotated[list[dict], operator.add]
    compressed_docs: list[dict]
    draft: NotRequired[str]
    answer: str
    retry_count: int
```

A node's return value is a *partial* update — `{"draft": text}"`, not the entire state — and LangGraph merges it into the running state object after the node completes. Declaring the full shape once, centrally, is what makes it possible to look at one file and know everything any node in a large graph is allowed to touch.

19.4 Annotated[list[dict], operator.add] — reducers for fan-out and fan-in

> **Definition — Reducer**
> A function attached to a state field via `Annotated[type, reducer_fn]` that combines an incoming partial update with the field's existing value, instead of the update simply overwriting it — the mechanism that lets multiple parallel branches all write to the same field safely.

Without a reducer, a field's default merge behavior is overwrite: the last branch to finish wins, and every other branch's contribution to that field is silently lost. `retrieved_document_chunks` is fed by every parallel `retrieve` call spawned from a fan-out (Section 19.6) — overwrite semantics would keep only the last variant's chunks. `operator.add` (list concatenation) instead accumulates every branch's contribution into one combined list.

```
retrieved_document_chunks: Annotated[list[dict], operator.add]
variants_with_chunks:      Annotated[list[dict], operator.add]
newly_failed_variants:     Annotated[list[str],  operator.add]
```

All three of Memora's list-accumulating fields use the same reducer for the same reason: each is written by every parallel retrieval branch, and the graph needs all of their contributions, not just the last one to finish.

19.5 NotRequired[…] — fields that may or may not appear

> **Definition — NotRequired field**
> A `GraphState` field marked optional via `typing.NotRequired`, signaling that a given run may complete without ever setting it — read with `state.get(field, default)`, never `state[field]`, since indexing would raise a `KeyError` on a run where it was never populated.

Memora's `switches` field is the clearest case: a per-request dictionary of `ENABLE_*` overrides that most requests never set at all. Its own docstring states the contract plainly — a request without overrides transparently falls back to `config.py` defaults, precisely because the field is optional rather than defaulted to an empty dict that would need constructing on every request regardless of whether anything overrides it.

```
switches: NotRequired[dict[str, bool]]
blocked_variants: NotRequired[list[str]]
draft: NotRequired[str]
quality_verdict: NotRequired[str]
```

`draft` and `quality_verdict` are optional for a related but distinct reason: a run that skips draft creation entirely (Section 19.7's routing) never writes either field, and every node downstream that might read them does so through `.get()` with an explicit fallback, never assuming they exist.

19.6 A minimal decide, retrieve, generate workflow

Strip Memora's real graph down to its smallest honest version and three nodes remain: decide whether the accumulated evidence is enough, retrieve more if not, generate an answer if so.

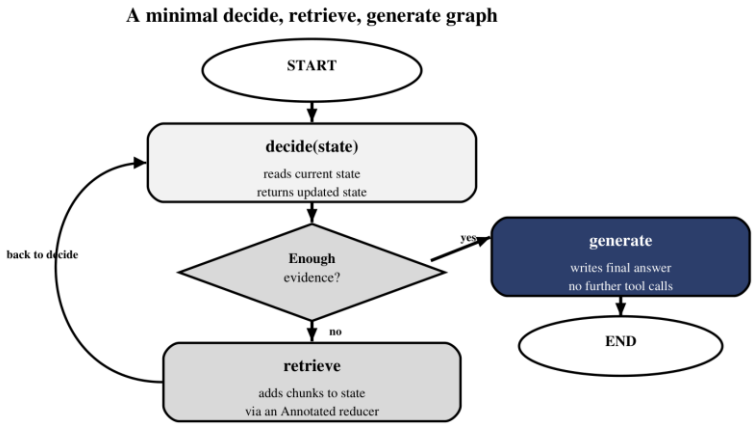

Figure 19.1 — The same decide-retrieve-generate loop Chapter 16 designed, now expressed as three nodes and one conditional edge.

Figure 19.1 is the graph form of exactly the loop Chapter 16 designed conceptually and Chapter 18 built by hand. Nothing about the underlying logic changed — `decide` still reads state and chooses; `retrieve` still adds evidence; `generate` still writes the final answer once. What changed is that the loop-back edge and the exit condition are now declarations the framework can inspect, log, and visualize, rather than a `while` condition and a `continue` statement buried in a function body.

19.7 Conditional edges and routing functions

A routing function's contract is deliberately narrow: read the state, return a string naming the next node, and do nothing else — no side effects, no state mutation. Keeping routing logic this pure is what makes a graph's control flow legible from `graph.py` alone, without having to read every node's implementation to know what can happen next.

```
def route_after_quality_check(state: GraphState) -> str:
    if state.get("quality_verdict") == QUALITY_PASS_VERDICT:
        return "generate_answer"
    budget_ok = (
        get_switches(state)["ENABLE_SUB_QUERY_GENERATION"]
        and state.get("retry_count", 0) < LLM_RESPONSE_RETRY_LIMIT
    )
    return "retry" if budget_ok else "generate_answer"
```

This is the exact same budget-with-headroom logic Chapter 18.5 implemented inline inside `run_agent`'s JUDGE phase — reproduced here as a standalone function with no access to anything but the state dictionary it was handed. Moving control flow out of node bodies and into named, single-purpose routing

functions is what lets `graph.py`'s edge declarations read as a table of contents for the entire pipeline's behavior.

19.8 Visualizing the graph

A compiled graph can draw itself. `app.get_graph().draw_mermaid_png()` walks every registered node and edge and renders an actual PNG — not a hand-maintained diagram that drifts out of sync with the code, but a direct visualization of whatever `graph.py` currently declares.

```
app = graph.compile()
png_bytes = app.get_graph().draw_mermaid_png()
Path("rag_graph.png").write_bytes(png_bytes)
```

> **Common pitfall — Calling .get_graph() on the uncompiled StateGraph**
> `StateGraph` and its compiled form (`graph.compile()`) are different objects with different capabilities — visualization and execution both require the compiled `app`, not the builder you called `add_node` on. This exact mistake was one of the three that surfaced during Memora's own first hands-on LangGraph session (Section 19.1); it produces a confusing error rather than an obviously wrong result, which is what makes it worth naming explicitly rather than trusting it will be self-evident.

Regenerate this diagram whenever `graph.py` changes and treat a stale copy as worse than no diagram at all — a visualization that silently drifted out of sync with the actual edges is a trap for exactly the debugging session it was supposed to help with.

19.9 Debugging a LangGraph flow

A graph's execution is harder to follow with a debugger than a linear function's — control genuinely jumps between independent functions rather than flowing top to bottom — which makes deliberate, structured logging at node boundaries load-bearing rather than optional. Every node in Memora's port logs its own inputs and outputs on entry and exit, and a project-specific tracing decorator, `instrument_namespace`, is applied to every node and service module specifically to keep that logging consistent without hand-writing it at every call site.

```
from services.operation_tracing import instrument_namespace as
_instrument_namespace
_instrument_namespace(globals(), "Retrieval Node", exclude={"retrieve"})
```

- Log each node's relevant inputs and outputs at entry and exit, not just on failure.
- Snapshot state at a graph's known trouble points — after a fan-in barrier is the highest-value place to look first.
- Set an explicit recursion limit; a routing bug that loops two nodes against each other fails loudly against a limit instead of hanging silently.
- Read the compiled graph's visualization (Section 19.8) before assuming a routing function's logic — the drawn graph shows what the framework will actually do, not what you intended.

19.10 When LangGraph helps and when an imperative loop is enough

Memora's own research settled this question with a criterion sharper than "does the pipeline have multiple phases" — nearly every pipeline does. The real question is whether the pipeline has independent phases that would genuinely parallelize, failure modes an existing retry path cannot already recover from, or long-running pauses — human review, hours-long waits — that need to survive a process restart.

Pipeline shape	LangGraph's benefit	Verdict
Strict sequential dependency chain (NAC then DC then LBC)	None — runtime equals a plain loop, plus framework overhead	An imperative loop is enough
Independent branches that could run concurrently (two compression tracks)	Real speedup — `max(branches)` instead of `sum(branches)`	Graph orchestration earns its cost
In-process transient failure (a retryable timeout)	Marginal — a scripted retry already handles this	Framework checkpointing adds little
Failure that must survive a process crash or deployment	Durable checkpointing recovers from the last completed node	Graph orchestration earns its cost

Checkpointing deserves its own honest caveat: an in-memory checkpointer only helps within a single live process, and durability past a crash requires writing checkpoints somewhere persistent — SQLite, Postgres, a file store — before the failure happens. LangGraph does not make the LLM calls, embedding calls, or vector-store queries underneath it any faster; it only changes what happens around a failure, and only where a failure mode existed for it to help with in the first place.

19.11 Non-barrier fan-in — the default behavior

A node with more than one incoming edge does not wait for all of its predecessors by default — it runs once per predecessor that reaches it, in whatever order they arrive. This is the correct behavior when a join node's job is genuinely per-branch (log each branch as it finishes, for instance), and a silent correctness bug when the node's job actually depends on having every branch's output at once.

> **Common pitfall — Assuming a multi-predecessor node runs exactly once**
> Memora's two compression tracks have unequal depth — documents run NAC then DC then LBC, three stages; learned_qa runs DC then LBC, two stages — so they finish in different graph supersteps even when started together. A join node registered without a barrier would fire once when the shorter track's edge arrives, using whichever track happened to be ready and treating the other as absent, and fire again when the second edge arrives. Section 19.12 is the fix.

19.12 The defer=True flag

> **Definition — defer=True**
> A `graph.add_node(...)` option that registers a node as a true fan-in barrier — LangGraph holds it until every other pending task in the current execution has completed, then runs it exactly once, regardless of how many predecessor edges feed it or how unevenly deep those predecessors' paths are.

Figure 19.2 shows exactly the shape Section 19.11 warned about: two tracks of unequal depth, both required, joined by a single barrier.

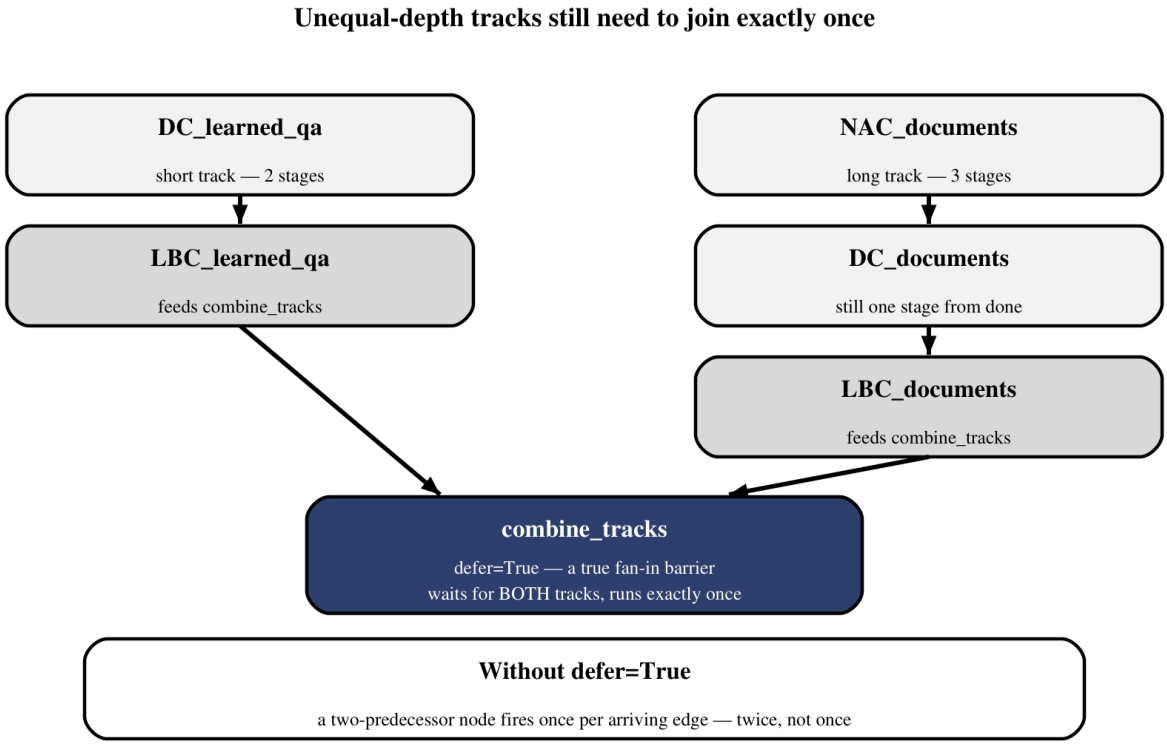

Figure 19.2 — Two tracks of different depth, one barrier, one execution.

```
graph.add_node("combine_tracks", combine_tracks, defer=True)
```

The comment beside this single line in Memora's `graph.py` is worth reading verbatim, because it states the reasoning as precisely as the mechanism itself: the two tracks land in different supersteps, so a plain multi-predecessor node would fire once per predecessor instead of once total, and `defer=True` is what forces genuine barrier semantics instead. One keyword argument, and an entire class of race-shaped bug becomes structurally impossible rather than merely unlikely.

19.13 Reducer semantics revisited — combining Annotated with defer=True

Section 19.4's reducers and Section 19.12's barrier solve two different halves of the same problem, and Memora's graph needs both, at two different points, for two different reasons. `retrieved_document_chunks` and `retrieved_learned_qa_chunks` accumulate through `operator.add` as an arbitrary number of parallel `retrieve` branches — one per query variant — each contribute their own chunks; nothing waits for a fixed count, because the fan-out itself (Section 19.6's `Send`) determines how many branches exist. `combine_tracks`, by contrast, has exactly two, always-present predecessors of unequal depth, and needs all of both, every time, with no accumulation logic beyond simple concatenation once both arrive.

Mechanism	Solves	Used where
`Annotated[list, operator.add]`	Combining an unknown or variable number of contributions	Fan-out over query variants
`defer=True`	Guaranteeing a fixed set of predecessors all finish before running	Fan-in after two asymmetric-depth tracks

Reach for a reducer when a field's *number* of writers varies; reach for `defer=True` when a node's *set* of predecessors is fixed but their timing is not. Confusing the two — relying on a reducer to somehow wait for stragglers, or adding `defer=True` to a node whose writer count genuinely varies — solves neither problem correctly.

19.14 No-code comparison — testing cyclic execution in Langflow

A parallel, no-code experiment tested the same underlying question — can a visual flow builder execute a real cycle — in Langflow, a drag-and-drop LLM pipeline tool. The answer was yes, through a dedicated `Loop` component providing an explicit back-edge and a guaranteed stop condition: fed a three-row CSV, the looped section executed once per row and terminated deterministically when the rows were exhausted.

A second experiment then exposed a distinction worth carrying back into graph-based thinking generally: the loop reused a single captured chat message across every iteration rather than prompting the user again per row, because Langflow's `Chat Input` captures one message at flow start, not a per-iteration `input()` call. The project's own research notes name this precisely — an *automatic data cycle* (what the `Loop` component provides) is not an *interactive cycle* (pause, wait for a new message, resume), and the two are easy to conflate until a test run reveals which one you actually built.

The comparison went one step further: the entire Memora agentic RAG pipeline was ported into Langflow as a generated Custom-Component flow, and the port immediately surfaced a real bug worth knowing about regardless of which tool produced it — Langflow's Knowledge node returns Chroma's raw `similarity_search_with_score` value (negative distance, roughly in `[-2, 0]`, closer to `0` meaning better) where Memora's own retriever returns a true `[0, 1]` cosine similarity. Every ported threshold, written against the `[0, 1]` scale, rejected every retrieved row until the mismatch was found — the same distance-versus-score confusion Chapter 11.4's `1 - distance` pitfall warned about, reappearing across a tool boundary instead of a distance-metric boundary.

A no-code tool reaching the same cyclic-execution conclusion as a hand-coded graph framework is not a coincidence — both are converging on the same underlying requirement: bounded, explicit, inspectable iteration, whichever surface expresses it. Chapter 19B now takes every primitive from this chapter and ports Chapter 18's exact agent onto them, node by node, so the comparison stops being abstract.